

CURLIX

Scalable Architecture Design

Shorten smarter. Track everything.

PRD v2.0 — Anonymous Free Stack MVP

Document Type: Architecture Design Report | Infrastructure: 100% Free Tier

Prepared: May 2026

Architecture Summary: Curlix is a stateless, horizontally scalable URL shortener built entirely on zero-cost infrastructure. It uses a Redis-first cache layer for sub-10ms redirects, async BullMQ analytics queues to decouple writes from the hot redirect path, anonymous bcrypt-hashed ownership tokens, and Cloudflare Turnstile for bot protection.

1. System Architecture Overview

The system is composed of five logical layers, each independently deployable and collectively designed to keep the critical redirect path as fast as possible.

Layer	Technology	Role
Client layer	React + Vite / Vercel Hobby	Global CDN, free SSL, automatic preview deployments on PR.
Edge / protection layer	Cloudflare Turnstile + Helmet.js	CAPTCHA verification, rate limiting shared across instances via Redis, security headers on all responses.
API layer	Node.js + Express / Render Free	Stateless REST API. Horizontally scalable. Never blocks redirect on analytics writes.
Data layer	PostgreSQL (Supabase) + Redis (Upstash)	Redis as L1 cache for redirects. PostgreSQL as source of truth with cache-miss fallback. Analytics in append-only table.
Async worker	BullMQ / Upstash Redis queue	Drains analytics jobs from queue and writes to PostgreSQL asynchronously. Can scale independently from API.

Key architectural decision — async analytics:

The redirect endpoint (`GET /:code`) immediately fires a BullMQ job and returns the 302. It never waits for an analytics DB write. This ensures redirect latency is bounded only by the Redis lookup, typically under 5ms, regardless of analytics queue depth or worker availability.

2. Database Schema

Two tables. The **urls** table is the hot read path — optimized for short_code lookups. The **analytics** table is append-only and read only via aggregation queries, keeping it completely isolated from the redirect critical path.

urls — source of truth

Column	Type	Notes / Constraints
id	uuid	PK, gen_random_uuid(), immutable
short_code	varchar(30)	UNIQUE NOT NULL, B-TREE indexed, nanoid-generated or custom alias
long_url	text	NOT NULL, validated against SSRF blocklist at write time
owner_token_hash	text	bcrypt hash of raw token (saltRounds=12), never raw token stored
custom_alias	varchar(50)	nullable, UNIQUE, sanitized regex [a-zA-Z0-9_-]
created_at	timestampz	NOT NULL DEFAULT now()
expires_at	timestampz	nullable, checked on every redirect, cached value includes expiry
is_active	boolean	NOT NULL DEFAULT true, soft delete flag, indexed

```
Indexes on urls: CREATE UNIQUE INDEX idx_urls_short_code ON urls(short_code); CREATE INDEX idx_urls_expires ON urls(expires_at) WHERE expires_at IS NOT NULL; CREATE INDEX idx_urls_active ON urls(is_active);
```

analytics — append-only click log

Column	Type	Notes / Constraints
id	uuid	PK
url_id	uuid	FK → urls.id ON DELETE CASCADE, NOT NULL
clicked_at	timestampz	NOT NULL DEFAULT now(), indexed for time-series range scans
referrer	text	nullable, raw Referer header value
user_agent	text	raw UA string, parsed by worker into device_type
device_type	varchar(20)	desktop / mobile / tablet, populated by worker
country	char(2)	ISO 3166-1 alpha-2, from CF-IPCountry header
ip_hash	varchar(64)	SHA-256 of client IP — never store raw IP

Indexes on analytics: CREATE INDEX idx_analytics_url_time ON analytics(url_id, clicked_at DESC);
CREATE INDEX idx_analytics_time ON analytics(clicked_at DESC); Analytics queries always scope to a single url_id — full table scans never occur in production.

Relationship: urls (1) → analytics (∞). ON DELETE CASCADE ensures click records are removed when a link is hard-deleted. Soft deletes (is_active=false) preserve analytics history.

3. API Architecture

Five endpoints, two response patterns: synchronous for mutations, fire-and-return for redirects. All endpoints share Redis-backed rate limiting applied as middleware before route handlers.

POST

/api/shorten

Create short URL

Body: long_url, alias? (optional), expires_at? (optional) • Auth: Cloudflare Turnstile token in CF-Turnstile-Response header • Rate limit: 10 req/min per IP • Returns: { short_url, owner_token } • Flow: validate URL → verify CAPTCHA → check alias uniqueness → write DB → prime Redis → return token (shown once only)

GET

/:code

Redirect to destination (hot path)

Returns: 302 Location: long_url • Cache: Redis L1 lookup first → cache hit: 302 immediately + BullMQ job enqueued async → cache miss: PostgreSQL lookup → warm Redis → 302 + enqueue job • Critical: analytics write is NEVER on the synchronous path. Redirect latency is bounded only by Redis.

GET

/api/analytics/:code

Aggregated click statistics

Auth: Authorization: Bearer {owner_token} — verified via bcrypt.compare() • Returns: { total_clicks, clicks_by_day[], device_breakdown[], referrers[] } • Query: always scoped to url_id — GROUP BY date_trunc('day', clicked_at), device_type

PATCH

/api/links/:code

Update destination URL or alias

Auth: Authorization: Bearer {owner_token} • Side effect: Redis key for :code is deleted (DEL) after successful DB write — never serve stale cache • Rate limit: 5 req/15min per IP (separate limit from creation)

DELETE

/api/links/:code

Soft-delete a link

Auth: Authorization: Bearer {owner_token} verified via bcrypt.compare() • Action: UPDATE urls SET is_active=false WHERE short_code=:code → DEL Redis key • Preserves analytics history. Link returns 404 on subsequent redirects.

Middleware stack (applied in order): helmet() → cors() → express.json() → rateLimiter (Redis) → captchaVerifier (creation only) → route handler

4. Authentication System

Curlix is anonymous-first — no sessions, no JWTs, no user accounts. Ownership is proven purely by possession of a cryptographically random token issued at link creation. The system never stores the raw token anywhere.

Token creation flow

1. User submits long URL	POST /api/shorten with Turnstile CAPTCHA token in header.
2. CAPTCHA verified server-side	POST to Cloudflare Turnstile API. Request rejected immediately if verification fails.
3. Owner token generated	nanoid(32) — 32 URL-safe random characters, ~191 bits of entropy.
4. Token hashed	bcrypt(rawToken, { saltRounds: 12 }). The cost factor makes brute force expensive even with DB access.
5. Hash persisted	owner_token_hash written to urls table. Raw token is never written anywhere.
6. Returned to user	{ short_url, owner_token: rawToken }. This is the only time the raw token is visible.

Token verification flow (edit / delete)

1. Client sends token	Authorization: Bearer {rawToken} header on PATCH or DELETE request.
2. Fetch DB record	SELECT owner_token_hash FROM urls WHERE short_code = :code AND is_active = true.
3. bcrypt.compare()	bcrypt.compare(rawToken, storedHash) — timing-safe comparison prevents enumeration.
4a. Match → proceed	Execute mutation (update or soft delete) then invalidate Redis key.
4b. No match → 401	Return generic 'Unauthorized' with no detail. Do not reveal whether the code exists.

Important UX note: The owner token is shown exactly once, at creation time. There is no recovery mechanism — this is intentional for a zero-signup system. The frontend must prominently prompt users to copy and save their token before navigating away.

5. Security Considerations

Threat	Severity	Mitigation
SSRF via long_url	High	Reject any URL resolving to RFC 1918 private ranges (127.0.0.0/8, 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) and IPv6 link-local (fe80::/10). Validate before touching cache or DB. Use the 'is-url' + 'url' Node packages for parsing, never regex alone.
Alias squatting / route collision	Medium	Maintain a reserved keyword blocklist covering all Express route prefixes (api, health, metrics, admin) plus single-character codes. Enforce regex <code>/^[a-zA-Z0-9_-]{2,50}\$/</code> on alias input. Check blocklist before uniqueness check.
Token bruteforcing	Medium	Separate Redis rate limit on PATCH/DELETE: 5 attempts per IP per 15 minutes. After 5 failed verifications on the same short_code, lock that code's mutate operations for 1 hour. bcrypt cost=12 adds ~250ms per check as additional bruteforce deterrent.
Analytics queue flooding	Low-Medium	BullMQ concurrency cap of 5 workers. Job TTL of 24 hours so spikes don't fill Upstash free-tier storage. Monitor queue depth — if consistently > 1000 jobs, add a second worker process.
Redirect chain abuse	Low	Do not follow redirects when validating long_url. Accept only the submitted URL at face value. If the destination later becomes malicious, that is a content moderation problem, not a validation gap — out of scope for MVP.
Header security	Low	Helmet.js defaults cover most headers. Explicitly add Referrer-Policy: strict-origin-when-cross-origin to prevent leaking destination URLs in the Referer header during redirects. Content-Security-Policy should be set on the frontend separately via Vercel headers config.
IP privacy in analytics	Low	Never store raw client IP. Store SHA-256(ip + daily_salt) where daily_salt rotates each midnight. This enables click deduplication within a day without enabling long-term tracking.

6. Deployment Strategy

All services run on zero-cost free tiers. The deployment is designed to be production-grade within the constraints of each free tier, with UptimeRobot as a keep-alive mechanism for Render's cold-start behavior.

Service topology

Service	Platform	Tech	Notes
Frontend	Vercel Hobby	React + Vite SPA	Automatic deploys on push to main. Preview URLs on every PR. Free SSL via Vercel. Configure VITE_API_BASE_URL as an environment variable pointing to Render URL.
API + Worker	Render Free	Node.js + Express	Single Render web service running both Express API and BullMQ worker in the same process. Add a /health endpoint returning 200 with no DB/Redis calls — used by UptimeRobot. Render spins down after 15 min inactivity — UptimeRobot 5-min ping prevents this.
Database	Supabase Free	PostgreSQL 15	500 MB storage, 2 GB data transfer/month. Enable connection pooling via Supabase's built-in pgBouncer (transaction mode) — Render free tier has limited concurrent connections.
Cache + Queue	Upstash Free	Redis	10,000 requests/day on free tier. Redis serves both as L1 redirect cache and BullMQ queue backend. Monitor daily request count — cache hit ratio should keep this well within limits at MVP scale.
Bot protection	Cloudflare Turnstile	CAPTCHA	Free for unlimited verifications. Widget renders client-side, token verified server-side via POST https://challenges.cloudflare.com/turnstile/v0/siteverify .
Keep-alive	UptimeRobot Free	HTTP monitor	Ping GET /health on Render URL every 5 minutes. Prevents cold starts. Free tier allows 50 monitors — more than sufficient.

CI/CD and environment management

GitHub is the single source of truth for code. Vercel and Render both integrate directly with GitHub repositories for automatic deployments. A simple GitHub Actions workflow runs on every push to main to validate that all required environment variables are present in the Render deployment before traffic is routed to the new instance.

Variable	Purpose
<code>DATABASE_URL</code>	Supabase PostgreSQL connection string with pgBouncer port (6543)
<code>UPSTASH_REDIS_URL</code>	Upstash Redis REST URL — used by both cache and BullMQ
<code>UPSTASH_REDIS_TOKEN</code>	Upstash Redis REST token
<code>TURNSTILE_SECRET_KEY</code>	Cloudflare Turnstile server-side secret
<code>BCRYPT_SALT_ROUNDS</code>	12 — set explicitly, never rely on default
<code>BASE_URL</code>	Public Render URL, used to construct short URLs in responses

Variable	Purpose
<code>NODE_ENV</code>	production

7. Key Architectural Decisions

Short code generation

`nanoid(8)`

~281 trillion combinations, URL-safe alphabet, collision probability negligible at MVP scale. No DB round-trip needed before insert — use `INSERT ... ON CONFLICT` to handle the rare collision.

Cache invalidation strategy

`Explicit DEL on write`

Simpler than TTL-based invalidation and immediate. On `PATCH` or `DELETE`, always `DEL` the Redis key in the same operation as the DB write. Never serve stale destination after an update.

Redirect HTTP status

`302 Found (not 301)`

301 is cached by browsers permanently — breaks analytics (clicks stop being recorded) and makes destination edits invisible to returning visitors. 302 is the correct choice for a trackable shortener.

Analytics decoupling

`BullMQ async queue`

Redirect latency is bounded by Redis, not DB write throughput. The worker is a separate concern and can be scaled, paused, or replaced without touching the redirect path.

Token storage

`bcrypt hash only`

A leaked DB dump cannot expose tokens that could be used to mass-edit or delete links. `bcrypt`'s cost factor provides time-hardening against offline attacks.

Soft delete

`is_active = false`

Preserves analytics history for the lifetime of the link. Allows potential future undelete feature. Hard delete available as an admin operation only.

IP privacy

`SHA-256 + daily salt`

Enables same-day deduplication without enabling cross-day tracking. Complies with GDPR's data minimization principle without requiring user consent flows.

Connection pooling

`Supabase pgBouncer`

Render free tier has limited persistent connections. `pgBouncer` in transaction mode multiplexes many application connections onto few PostgreSQL connections, preventing connection exhaustion.